

Chapter 2



The Relational Model



The Relational Model



- 2.1 The CAP Database**
- 2.2 Naming Part of a Database**
- 2.3 Relational Rules**
- 2.4 Keys, Superkeys, and Null Values**
- 2.5 Relational Algebra**
- 2.6 Set-Theoretic Operations**
- 2.7 Native Relational Operations**
- 2.8 The Interdependence of Operations**
- 2.9 Illustrative Examples**
- 2.10 Other Relational Operations**

2.1 The CAP Database



- The **CAP** database (for the table names **CUSTOMERS**, **AGENTS**, and **PRODUCTS**)
- The CAP database is used by a wholesale business to keep track of its *customers*, the *products* it sells to these customers, and the *agents* who place *orders* for products on behalf of these customers.

CAP Database



CUSTOMERS

cid	cname	city	discent
c001	TipTop	Duluth	10.00
c002	Basics	Dallas	12.00
c003	Allied	Dallas	8.00
c004	ACME	Duluth	8.00
c006	ACME	Kyoto	0.00

AGENTS

aid	aname	city	percent
a01	Smith	New York	6
a02	Jones	Newark	6
a03	Brown	Tokyo	7
a04	Gray	New York	6
a05	Otasi	Duluth	5
a06	Smith	Dallas	5

PRODUCTS

pid	pname	city	quantity	price
p01	comb	Dallas	111400	0.50
p02	brush	Newark	203000	0.50
p03	razor	Duluth	150600	1.00
p04	pen	Duluth	125300	1.00
p05	pencil	Dallas	221400	1.00
p06	folder	Dallas	123100	2.00
p07	case	Newark	100500	1.00

CAP Database

ORDERS



ordno	month	cid	aid	pid	qty	dollars
1011	jan	c001	a01	p01	1000	450.00
1012	jan	c001	a01	p01	1000	450.00
1019	feb	c001	a02	p02	400	180.00
1017	feb	c001	a06	p03	600	540.00
1018	feb	c001	a03	p04	600	540.00
1023	mar	c001	a04	p05	500	450.00
1022	mar	c001	a05	p06	400	720.00
1025	apr	c001	a05	p07	800	720.00
1013	jan	c002	a03	p03	1000	880.00
1026	may	c002	a05	p03	800	704.00
1015	jan	c003	a03	p05	1200	1104.00
1014	jan	c003	a03	p05	1200	1104.00
1021	feb	c004	a06	p01	1000	460.00
1016	jan	c006	a01	p01	1000	500.00
1020	feb	c006	a03	p07	600	600.00
1024	mar	c006	a06	p01	800	400.00

2.2 Naming the parts of a Database



- Two different standard terminologies are used in databases.
 - One terminologies refers to *tables*, *columns*, and *rows*
 - Other refers to *relations* (the counterpart of *tables*), *tuples/records* (corresponding to *rows*), and *attributes* (for *columns*)

2.2 Naming the parts of a Database



Name of relation(table) $\longrightarrow$ student

number	Name	Age	Sex	Depart	grade
95004	张三	19	男	社会学	98
95006	李四	20	男	心理学	98
95008	王五	18	男	法律学	98
...	...	...	...	...	...

Tuple (row)

Attribute (column)

分量Data Element:
元组中的一个属性值

2.2 Naming the parts of a Database



- The CAP database consists of the **set** of tables:
 - ✓ $CAP = \{CUSTOMERS, AGENTS, PRODUCTS, ORDERS\}$
- We define the **heading** of a table to be a set of columns whose names appear at the top of the table
 - ✓ $Head(CUSTOMES) = \{cid, cname, city, discnt\}$

Domain and Datatype

- **domain** is usually defined in such cases to contain the exact set of constants that can appear as values for the attribute (enumerate 枚举).
- Each column in the table is declared to have a certain **type**.

number	Name	Age	Sex	Depart	grade
95006	李四	20	男	心理学	98
95008	王五	18	男	法律学	98
...	...	...	...	...	...

domain (sex)
= (男, 女)

2.3 Relational Rules 关系规则



➤ Rules of the relational model tell us what variations in table structure are permitted and limit possible retrieval operations.

RULE 1: First Normal Rule. In defining tables, the relational model insists that columns that have *multi-valued attributes* (sometimes called repeating fields) or *have any internal structure* (like a record) are not permitted! A table that obey this is said to be in first normal form.

RULE 1: First Normal Rule

Example:

EMPLOYEES			
eid	ename	position	dependents
e001	Smith, John	Agent	Michael J.
			Susan R.
e002	Andrews, David	Superintendent	David M. Jr.
e003	Jones, Franklin	Agent	Andrew K.
			Mark W.
			Louisa M.

图2-3 具有多值属性列dependents 的EMPLOYEES 表

RULE 1: First Normal Rule



Example:

EMPLOYEES					
eid	ename	position	dependent1	dependent2	...
e001	Smith, John	Agent	Michael J.	Susan R.	...
e002	Andrews, David	Superintendent	David M. Jr.	...	...
e003	Jones, Franklin	Agent	Andrew K.	Mark W.	...

图2-4 每个雇员行中具有多列家属名的EMPLOYEES 表

RULE 1: First Normal Rule



Example:

EMPLOYEES			DEPENDENTS	
eid	ename	position	eid	dependent
e001	Smith, John	Agent	e001	Michael J.
e002	Andrews, David	Superintendent	e001	Susan R.
e003	Jones, Franklin	Agent	e002	David M. Jr.
			e003	Andrew K.
			e003	Mark W.
			e003	Louisa L.

图2-5 表EMPLOYEES 和相应的表DEPENDENTS

2.3 Relational Rules



RULE 2: Access Rows by Content Only Rule.

A second rule of relational model states that we can only retrieve rows by their content, the attribute values that exist in each row.

(A relation is a **set** of tuples)

——This rule *is broken* by the standard SQL language, as we will see in Chapter 3.

2.3 Relational Rules



RULE 3: The Unique Row Rule. A third rule of the relational model is that two tuples in a relation (row in a table) cannot be identical完全同样的 in all column values at once.

(a relation can be thought of a **set** of tuples, and a set never contains two identical elements)

All these rules mean that we couldn't simply define tables to hold the data in, and issues of database design are the same in all cases.

六条性质：

- 每一列中的分量是同一类型的数据，来自同一个域。
- 不同的列可以出自同一个域，称其中的每一列为一个属性，不同的属性要给予不同的属性名。
- 列的顺序无所谓，即列的次序可以任意交换。
- 行的顺序无所谓，即行的次序可以任意交换。
- 任意两个元组不能完全相同。
- 分量必须取原子值，即每一个分量都必须是不可分的数据项。

2.4 Keys, Superkeys, and Null Values

Definition 2.4.1: Table *Key*. Given a table T , with $\text{Head}(T)=A_1 \dots A_n$. A key for the table T , sometimes called a candidate key, is a set of attributes, $K=A_{i_1} \dots A_{i_k}$, with two properties:

- [1] ... (the value any row u takes on for the set of attributes K are unique)
 - [2] No proper subset H of K has property 1.
- (we refer to a set of attributes that fulfills property 1 but not necessarily property 2 as a *superkey*)

2.4 Keys, Superkeys, and Null Values

键? (唯一确定一个元组)

学号	姓名	年龄	性别	系别	年级
95004	张三	19	男	社会学	98
95006	李四	20	男	心理学	98
95008	王五	18	男	法律学	98
...	...	...	...	...	...

键? (唯一确定一个元组)

键? (唯一确定一个元组)

2.4 Keys, Superkeys, and Null Values

Example:

T

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d1
a2	b2	c1	d1
a2	b1	c2	d1

- 1) 属性D以及任何包含D的属性集合不可能是键
- 2) 属性集合AB、AC、BC、ABC、ABD、ACD、BCD、ABCD是超键
- 3) 属性集合AB、AC、BC是键

THEOREM 2.4.2: Every table T has at least one key

2.4 Keys, Superkeys, and Null Values

Definition 2.4.3: *Primary Key* of a Table. A primary key of a table T is the candidate key chosen by the database designer to uniquely identify specific rows of T.

■ **Primary Key**（主键）和**Foreign Key**（外键）

- 在关系数据库系统中，表与表之间的关联件是由 **key**（键）来建立的。**Primary key**(主键)用来标识记录的字段，具有唯一性，不允许重复。**Foreign key**(外键)用来建立与其他表的联系。

2.4 Keys, Superkeys, and Null Values

书籍编号	书名	作者编号	单价	...
10001	DB2入门与提高	100	39.00	...
10002	SQL Server 数据库系统基础	101	38.00	...
...	...	...		...

主键

外键

作者编号	作者名	作者单位	职称	...
100	刘耸柏	清华大学	教授	...
101	罗运模等	中国人民大学	教授	...
...	...	...		...

主键

2.4 Keys, Superkeys, and Null Values



➤ Null Value:

- The null value used here should be interpreted as unknown or not yet defined, meaning that when we know more we intend to fill in the value
- has different properties than those of the number 0 or a blank or null string

RULE 4: Entity Integrity Rule. No column belonging to a primary key of a table T is allowed to take on ***null values*** for any row in T.

关系模型概述

- ❑ 关系模型由关系数据结构、关系操作集合和关系完整性约束三部分组成：

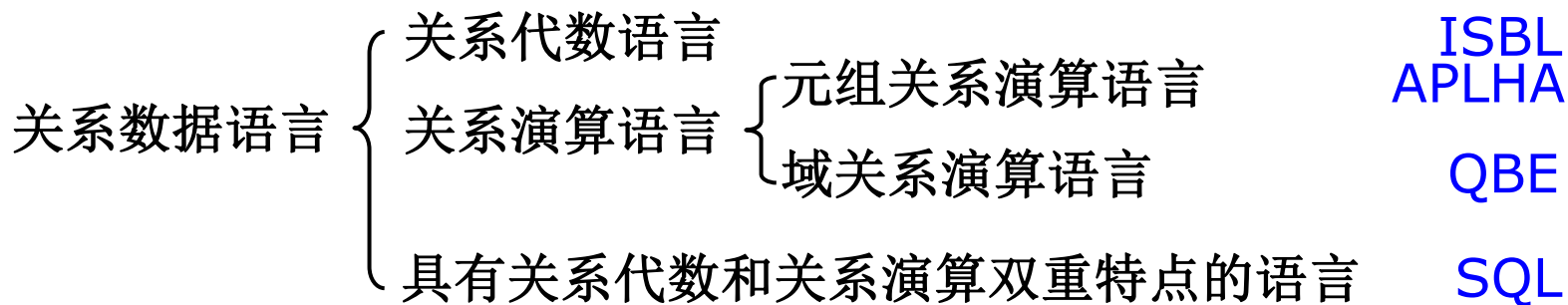
① 关系数据结构

- 在关系模型中，现实世界的实体以及实体间的各种联系均用关系来表示。在用户看来，关系模型中数据的逻辑结构是一张二维表。

关系模型概述

② 关系操作

- 包括：select(选择)、project(投影)、join (连接)、union (并)、divide(除)、intersection(交)、difference (差)等 query(查询)操作和 insert(插入)、delete(删除)、update(修改)操作两大部分。
- 关系操作的特点是集合操作方式，即操作对象和结果都是集合。这种操作方式也称为一次一集合 (**set-at-time**) 的方式。非关系数据模型的数据操作方式则称为一次一记录 (**record-at-time**) 方式。



关系模型概述

③ 关系三类完整性约束

- 关系模型允许定义三类完整性约束：**Entity Integrity(实体完整性)**、**Referential Integrity(参照完整性)**和**User_defined Integrity(用户定义的完整性)**。
 - 实体完整性和参照完整性是关系模型必须满足的完整性约束条件，应该由系统自动支持。
 - 用户定义的完整性是应用领域要遵循的约束条件，体现了具体领域中的语义约束。

Entity Integrity

- 实体完整性规则规定基本关系的所有主键都不能取空值。
- 对于实体完整性规则说明如下：
 - A. 实体完整性规则是针对基本关系而言的。一个基本表通常对应现实世界的一个实体集。
 - B. 现实世界的实体是可区分的，即它们具有某种唯一性标识。
 - C. 相应地，关系模型中主键应作为唯一性标识。
 - D. 主键的属性（值）不能取空值。

Referential Integrity

- 参照完整性规则规定两个相关联的表之间的主键和外键必须保持一致性，不能因为一个表的记录改变时，造成另一个表的内容变成无效的值。

参照完整性具有以下特征：

- A. 外键取空值。
- B. 或者等于参照表中的某个元组的主键值。
- 下面我们来看学生关系和专业关系。
 - 学生（学号、姓名、性别、专业号、年龄）
 - 专业（专业号、专业名）
- 对于学生关系中每个元组的“专业号”属性只能取下面两类值：
 - 空值：表示尚未给学生分配专业。
 - 非空值：这时该值必须是专业关系中某个元组的“专业号”值，表示该学生不可能分配到一个不存在的专业中。即被参照关系“专业”中一定存在一个元组，它的主键值等于该参照关系“学生”中的外键值。

∞ User_defined Integrity

- 用户定义的完整性是针对某一具体关系数据库的约束条件。它反映某一个具体应用所涉及的数据必须满足的语义要求。例如某个属性值必须取唯一值，某个属性的取值范围在**0~100**之间等，身份证号码的位数为**15—18**位。关系模型应提供定义和检验这类完整性的机制。

Exercises

P53

2.1, 2.2, 2.4

2.9, 2.10, 2.11, 2.14



2.5 Relational Algebra

- ✓ The aim was to demonstrate the potential for a query language to retrieve information from a relational database system.
- ✓ Information is stored in the form of tables, relational algebra can be thought of as a collection of method for building new tables that constitute answers to queries.
- ✓ It is an abstract language that can not be used on an actual computer, but it is useful in understanding SQL.

2.5 Relational Algebra

- ✓ **Fundamental Operations** of Relational Algebra
 - ✓ *set-theoretic operations* 集合运算
 - ✓ native relational operations 自然关系运算

集合运算			
名称	符号	键盘格式	示例
并	$\cup$	UNION	$R \cup S$, 或 $R \text{ UNION } S$
交	$\cap$	INTERSECT	$R \cap S$, 或 $R \text{ INTERSECT } S$
差	$-$	- 或 MINUS	$R - S$, 或 $R \text{ MINUS } S$
乘积	$\times$	TIMES	$R \times S$, 或 $R \text{ TIMES } S$

2.6 Set-Theoretic Operations



✓ **DEFINITION 2.6.1 Compatible Tables.**

Tables R and S are compatible if they have the same headings; that is, if $\text{Head}(R) = \text{Head}(S)$, with attributes chosen from the same domains and with the same meanings.

2.6 Set-Theoretic Operations



✓ *The Union, Intersection, and Difference Operations*

(1) **Union**: **R**和**S**的并是由属于**R**或属于**S**的元组组成的集合，记为

$$R \cup S \equiv \{t \mid t \in R \vee t \in S\}$$

(2) **Intersection**: **R**和**S**的交是由属于**R**又属于**S**的元组构成的集合，记为

$$R \cap S \equiv \{t \mid t \in R \wedge t \in S\}$$

(3) **Difference**: **R**和**S**的差是由属于**R**但不属于**S**的元组组成的集合，记为

$$R - S \equiv \{t \mid t \in R \wedge t \notin S\}$$

2.6 Set-Theoretic Operations



Example1:

R

A	B	C
a1	b1	c1
a1	b2	c3
a2	b1	c2

S

A	B	C
a1	b1	c1
a1	b1	c2
a1	b2	c3
a3	b2	c3

Example 2.6.3 $T := (R \cup S) - (R \cap S)$

2.6 Set-Theoretic Operations



✓ *The Product Operation*

设关系**R**和**S**的元数分别为**r**和**s**，定义**R**和**S**的乘积是一个**(r+s)**元的元组集合，每个元组的前**r**个分量来自**R**的一个元组，后**s**个分量来自**S**的一个元组，记为：

$$R \times S \equiv \{t \mid t = \langle t^r, t^s \rangle \wedge t^r \in R \wedge t^s \in S\}$$

Example1: $R \times S$

R	A	B	C
	a1	b1	c1
	a1	b2	c3
	a2	b1	c2

S	B	C	D
	b1	c1	d1
	b1	c1	d3
	b2	c2	d2
	b1	c2	d4

Example2: $R \times R$

2.5 Relational Algebra

- ✓ **Fundamental Operations** of Relational Algebra
 - ✓ set-theoretic operations 集合运算
 - ✓ *native relational operations* 自然关系运算

自然关系运算			
名称	符号	键盘格式	示例
投影	$R [\]$	$R [\]$	$R [A_1 \dots A_k]$
选择	$R \text{ where } C$	$R \text{ where } C$	$R \text{ where } A_1 = 5$
连接	$\bowtie$	JOIN	$R \bowtie S$, 或 $R \text{ JOIN } S$
除	$\div$	DIVIDE BY	$R \div S$, 或 $R \text{ DIVIDE BY } S$

2.7 Native Relational Operations



✓ *Precedence of Relational Operations*

优先级	运算	符号
高	投影	$R [\]$
	选择	$R \text{ where } C$
	乘积	$\times$
	连接、除法	$\bowtie, \div$
	交	$\cap$
低	并、差	$\cup, -$

图2-7 关系运算优先级

2.7 Native Relational Operations

✓ *The Projection Operation* 投影

The operation of **projection** acts on a table by deleting some of its columns.

Example 2.7.1: CN:=CUSTOMERS[cname]

CN

cname
Tiptop
Basics
Allied
ACME

2.7 Native Relational Operations



✓ *The Selection Operation* 选择

Create a table by selecting from a given table only those rows that satisfy a specified criterion

$S \text{ where } C$

- AND: $S \text{ where } C_1 \text{ and } C_2 \ (\cap)$
- OR: $S \text{ where } C_1 \text{ or } C_2 \ (\cup)$
- NOT: $S \text{ where not } C_1 \ (-)$

2.7 Native Relational Operations



Example 2.7.2:

- ✓ find all CUSTOMERS based in 'Kyoto'
customers where city='kyoto'

cid	cname	city	discent
c006	ACME	Kyoto	0.00

- ✓ find the products stored in Dallas that cost more than \$0.50
products where city = 'Dallas' and price > 0.50

2.7 Native Relational Operations



Example 2.7.5: suppose that we wish to determine those cities where we have either customers who have a discount of less than 10% or agent who make a commission of less than 6%.

$(\text{CUSTOMERS where } \text{discnt} < 10)[\text{city}] \cup (\text{AGENTS where } \text{percent} < 6)[\text{city}]$

2.7 Native Relational Operations



Example 2.7.3: 假设我们需要检索所有的代理对，他们都具有超过6%的佣金比例，并且都在同一个城市。

$L := \text{AGENT where percent} \geq 6$

$M := \text{AGENT where percent} \geq 6$

$\text{PAIRS} := (L \times M) \text{ where } L.\text{city} = M.\text{city} \text{ and } L.\text{aid} < M.\text{aid}$

PAIRS2

L.aid	L.aname	L.city	L.percent	M.aid	M.aname	M.city	M.percent
a01	Smith	New York	6	a04	Gray	New York	6

2.7 Native Relational Operations

✓ The Join Operation 连接

The purpose of the **join** operation is to create a table that relates the rows of two given tables that have equal values in identically named columns.

Example 2.7.6:

R		
A	B1	B2
a1	b1	b1
a1	b2	b1
a2	b1	b2

S		
B1	B2	C
b1	b1	c1
b1	b1	c2
b1	b2	c3
b2	b2	c4

R ⋈ S			
A	B1	B2	C
a1	b1	b1	c1
a1	b1	b1	c2
a2	b1	b2	c3

2.7 Native Relational Operations



✓ *The Join Operation* 连接

If the two tables R and S being joined **have no common attributes**, then the content of the join coincides with the content of the **product ($\times$)** of the two tables.

If R and S are compatible so that **all attributes are in common**, then the join of the two tables corresponds to the **intersection ($\cap$)** of the two tables.

2.7 Native Relational Operations



Example 2.7.7: find the names of the customers who have ordered product p01.

ORDERS where pid='p01'

(CUSTOMERS $\bowtie$ ORDERS where pid='p01')[cname]

Example 2.7.8: get names of customers who order at least one product costing \$0.50.

PRODUCTS where price=0.50

ORDERS $\bowtie$ (PRODUCTS where price=0.50)

ORDERS $\bowtie$ (PRODUCTS where price=0.50) $\bowtie$ CUSTOMERS

((ORDERS $\bowtie$ (PRODUCTS where price=0.50)[pid]) $\bowtie$ CUSTOMERS) [cname]

2.7 Native Relational Operations

✓ *The Division Operation* 除法

The table T is the result of the division $R \div S$, if $\text{Head}(T) = A_1 \dots A_n$ and T contains exactly those rows t such that for every row s in S, the row resulting from concatenating t and s can be found in table R.

Example 2.7.9

R

A	B	C
a1	b1	c1
a2	b1	c1
a1	b2	c1
a1	b2	c2
a2	b1	c2
a1	b2	c3
a1	b2	c4
a1	b1	c5

C
c1
c2

C
c1
c2
c3
c4

2.7 Native Relational Operations



Example:

R

A	B	C
a1	b1	c1
a2	b1	c1
a3	b2	c1
a1	b1	c2
a2	b2	c2
a3	b3	c2

S

C
c1
c2

- $(R \div S)[A]$
- $R[A, C] \div S$

2.7 Native Relational Operations



Example 2.7.10: 与顾客 **c006** 订购相同商品的所有顾客的顾客号

(1) $PC6 := (ORDERS \text{ where } cid = 'c006')[pid]$

(2) $CP := ORDERS[cid, pid]$

(3) $CP \div PC6$

c006 订购了什么商品??

PC6	
pid	
p01	
p07	

订单上所有顾客的订购记录??

所有订购了
'p01'和'p07'
商品的顾客的顾客号是??

2.7 Native Relational Operations



- whenever the word “all” is used in a retrieval request, the query expression to use may very well include the *division operation*.
- example2.7.11: get names of customers who order all products.
$$\text{ORDERS}[\text{cid}, \text{pid}] \div [\text{PRODUCTS}[\text{pid}]$$
$$((\text{ORDERS}[\text{cid}, \text{pid}] \div [\text{PRODUCTS}[\text{pid}]] \bowtie \text{CUSTOMERS}) [\text{cname}]$$

2.8 The Interdependence of Operations

basic operations: union, difference, product, selection, projection.

THEOREM 2.8.1 Let R and S be two compatible tables, where $\text{Head}(R) = \text{Head}(S)$. The intersection operation can be defined in terms of subtraction alone:

$$A \cap B = A - (A - B)$$

THEOREM 2.8.2 The join of two tables R and S , where $\text{Head}(R) = A_1 \dots A_n B_1 \dots B_k$ and $\text{Head}(S) = B_1 \dots B_k C_1 \dots C_m$ and $n, k, m \geq 0$, can be expressed using product, selection, and projection, together with assignment operator

$$T := (R \times S) \text{ where } R.B_1 = S.B_1 \dots R.B_k = S.B_k$$

2.9 Illustrative Examples



Example 2.9.1: get the names of customers who order at least one product priced at \$0.50.

(1) First, start with products priced at \$0.50

$(P \text{ where } price = 0.50)[pid]$

(2) Join with ORDERS, to find the customers who order the products

$(P \text{ where } price = 0.50)[pid] \bowtie O$

(3) have names of customers.

$(((P \text{ where } price = 0.50)[pid] \bowtie O) \bowtie C)[cname]$

2.9 Illustrative Examples



Example 2.9.2: find cids of customers who do not place any order through agent a03.

$(O \text{ where aid} = 'a03')[cid]$

$O[cid] - (O \text{ where aid} = 'a03')[cid]$

Example 2.9.3: retrieve customers who place orders *only through* agent a03.

$(O \text{ where aid} \neq 'a03')[cid]$

$O[cid] - (O \text{ where aid} \neq 'a03')[cid]$

2.9 Illustrative Examples



Example 2.9.4: find products have never been ordered by a customer based in Duluth through an agent based in New York.

$$P[\text{pid}] - ((C \text{ where city} = \text{'Duluth'})[\text{cid}] \cap O \cap (A \text{ where city} = \text{'New York'})[\text{aid}])[\text{pid}]$$

Example 2.9.5: get names of customers who order all products priced at \$0.50.

$$(P \text{ where price} = 0.50)[\text{pid}]$$
$$O[\text{cid}, \text{pid}] \div (P \text{ where price} = 0.50)[\text{pid}]$$
$$((O[\text{cid}, \text{pid}] \div (P \text{ where price} = 0.50)[\text{pid}]) \cap C)[\text{name}]$$

2.9 Illustrative Examples



Example 2.9.6: get cids of customers who order both products p01 and p07.

$(O \text{ where pid} = 'p01')[cid] \cap (O \text{ where pid} = 'p07')[cid]$

Example 2.9.7: get cids of customers who have the same discount as any customer in Dallas or Kyoto.

$(C \text{ where city} = 'Dallas' \text{ or city} = 'Kyoto')[discnt]$

$((C \text{ where city} = 'Dallas' \text{ or city} = 'Kyoto')[discnt] \bowtie C)[cid]$

2.10 Other Relational Operations



Next we discuss two additional useful relational operations that are not in the standard set, **outer join** and theta join θ – join , they make the job much simpler.

名称	符号	键盘格式	示例
外连接	$\bowtie_0$	OUTERJ	$R \bowtie_0 S$ 或 $R \text{ OUTERJ } S$
左外连接	$\bowtie_{L0}$	LOUTERJ	$R \bowtie_{L0} S$ 或 $R \text{ LOUTERJ } S$
右外连接	$\bowtie_{R0}$	ROUTERJ	$R \bowtie_{R0} S$ 或 $R \text{ ROUTERJ } S$
θ 连接	$\bowtie_{A_i \theta B_j}$	JN($A_i \theta B_j$)	$R \bowtie_{A_1 > B_2} S$ $R \text{ JN}(A_2 > B_2) S$

✓ **The outer join** in particular has significant ability to simplify queries in commercial database applications.

2.10 Other Relational Operations



Outer Join

To introduce the outer join we will use two tables, the **AGENTS** and the **SALES**.

AGENTS

aid	aname	city	percent
a01	Smith	New York	6
a02	Jones	Newark	6
a03	Brown	Tokyo	7
a04	Gray	New York	6
a05	Otasi	Duluth	5
a06	Smith	Dallas	5

SALES

aid	total
a01	850.00
a02	400.00
a03	3900.00
a05	2400.00
a06	900.00
a07	650.00

2.10 Other Relational Operations



Outer Join: print a table that provides all agent names, their aid, and their total sales.

$(\text{AGENTS} \bowtie \text{SALES})[\text{aname}, \text{aid}, \text{total}]$

AGENTS

aid	aname	city	percent
a01	Smith	New York	6
a02	Jones	Newark	6
a03	Brown	Tokyo	7
a04	Gray	New York	6
a05	Otasi	Duluth	5
a06	Smith	Dallas	5

SALES

aid	total
a01	850.00
a02	400.00
a03	3900.00
a05	2400.00
a06	900.00
a07	650.00

2.10 Other Relational Operations



Outer Join

$AGENTS \bowtie_O SALES[aname, aid, total]$

3) Outer Join

($\bowtie_O$ or $\boxminus$)

aname	aid	total
Smith	a01	850.00
Jones	a02	400.00
Brown	a03	3900.00
Gray	a04	null
Otasi	a05	2400.00
Smith	a06	900.00
null	a07	650.00

2.10 Other Relational Operations



Outer Join

$AGENTS \bowtie_{LO} SALES[aname, aid, total]$

1) Left Outer Join

($\bowtie_{LO}$ or $\Join$)

左外连接取出左侧关系中所有与右侧关系中任一元组都不匹配的元组，用空值null填充右侧关系的属性，构成新的元组，将其加入自然连接的结果中

aname	aid	total
Smith	a01	850.00
Jones	a02	400.00
Brown	a03	3900.00
Gray	a04	null
Otasi	a05	2400.00
Smith	a06	900.00

2.10 Other Relational Operations



Outer Join

$AGENTS \bowtie_{RO} SALES[aname, aid, total]$

2) Right Outer Join

($\bowtie_{RO}$ or $\bowtie_{\perp}$)

右外连接取出右侧关系中所有与左侧关系中任一元组都不匹配的元组，用空值null填充左侧关系的属性，构成新的元组，将其加入自然连接的结果中

aname	aid	total
Smith	a01	850.00
Jones	a02	400.00
Brown	a03	3900.00
Otasi	a05	2400.00
Smith	a06	900.00
null	a07	650.00

□ 关系代数的运算可分为两类

- 传统的集合运算：并、交、差、乘积
- 专门的关系运算：投影、选择、联接、除

□ 关系代数用到的运算符

- 集合运算符： $\cup$ (并)、 $-$ (差)、 $\cap$ (交)、 $\times$ (乘积)
- 专门的关系运算符： δ (选择)、 π (投影)、 $\bowtie$ (联接)、 $\div$ (除)
- 算术比较运算符： $\theta = \{>, >=, <, <=, =, \neq\}$
- 逻辑运算符： $\vee, \wedge, \neg$

relational algebra (review)



已知两个关系**R**和**S**，分别求出 **$R \cup S$** 、 **$R - S$** 、 **$R \times S$** 、

$\pi_{A, C}(R)$ 和 **$\delta_{B='b'}(R)$**

A	B	C
a	b	c
d	a	f
c	b	d

A	B	C
b	g	a
d	a	f

$R \times S$

R.A	R.B	R.C	S.A	S.B	S.C
a	b	c	b	g	a
a	b	c	d	a	f
d	a	f	b	g	a
d	a	f	d	a	f
c	b	d	b	g	a
c	b	d	d	a	f

$R \cup S$

A	B	C
a	b	c
d	a	f
c	b	d
b	g	a

$R - S$

A	B	C
a	b	c
c	b	d

$\pi_{A, C}(R)$

A	C
a	c
d	f
c	d

$\delta_{B='b'}(R)$

A	B	C
a	b	c
c	b	d

Exercises

P53

2.5, 2.6, 2.7, 2.8

2.12, 2.13

